

Guía Definitiva de C# para Prácticas en Espiral MS

Material de estudio personal

Julio 2026

Nivel: Transición Java → C# (Intermedio)

Duración estimada: 3-4 semanas, 5-8h/semana

Índice

1. [Introducción](#)
 2. [Lección 1: Sintaxis C#](#)
 3. [Lección 2: Colecciones y LINQ](#)
 4. [Lección 3: Async/Await](#)
 5. [Mini-proyecto: Gestor de Biblioteca](#)
 6. [Lección 4: Generics](#)
 7. [Lección 5: Delegates y Eventos](#)
 8. [Lección 6: Patrones de Diseño](#)
 9. [Lección 7: SQL](#)
 10. [Lección 8: Entity Framework](#)
 11. [Lección 9: Git](#)
 12. [Lección 10: Scrum](#)
 13. [Lección 11: Frontend](#)
 14. [Lección 12: TypeScript](#)
 15. [Resumen Final](#)
-

Introducción

[↑ Volver al índice](#)

Bienvenido a tu plan de capacitación en C# para las prácticas de DAM en Espiral MS. Este curso está diseñado específicamente para alguien que viene de Java y necesita estar preparado para un entorno profesional de .NET.

Lo que lograrás: - Dominar las diferencias clave entre Java y C# - Escribir código idiomático en C# (LINQ, async/await, events) - Entender patrones empresariales (Repository, Dependency Injection) - Estar listo para aprender Entity Framework y ASP.NET Core en las prácticas

Estructura del aprendizaje: - Semanas 1-2: Fundamentos de C# (sintaxis, colecciones, async) - Semana 2: Proyecto integrador (juntar todo en algo real) - Semana 3: Profundización (generics, delegates, patrones)

Lección 1: Sintaxis C# para quién viene de Java

[↑ Volver al índice](#)

1.1 Estructura básica de un programa

Java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hola mundo");  
    }  
}
```

C# (estilo clásico)

```
using System;  
  
namespace MyApp  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("Hola mundo");  
        }  
    }  
}
```

C# moderno (top-level statements, .NET 6+)

```
Console.WriteLine("Hola mundo");
```

Diferencias clave: - System.out.println → Console.WriteLine - C# usa namespace en vez de paquetes - Desde .NET 6, se permite código directo sin clase Program explícita - El punto de entrada Main puede ser static void Main() o static async Task Main() si necesitas await desde el arranque

1.2 Tipos y variables

Concepto	Java	C#
Inferencia de tipo	var x = 5; (Java 10+)	var x = 5;
Entero	int	int
Entero largo	long	long
Decimal	double	double
Decimal preciso	BigDecimal	decimal (recomendado para dinero)
Cadena	String	string (alias de System.String)
Carácter	char	char
Booleano	boolean	bool
Constante	final int X = 5;	const int X = 5;

Concepto	Java	C#
Solo-lectura tras construcción	No existe directo	readonly

Value types vs Reference types

Esta es una distinción más marcada en C# que en Java:

- **Value types** (int, double, bool, struct, enum): se copian por valor. Al asignar, se duplica el dato.
- **Reference types** (class, string, array, interface): se copian por referencia. Al asignar, ambas variables apuntan al mismo objeto.

```
// Value type: cada variable tiene su propia copia
int a = 5;
int b = a;
b = 10;
Console.WriteLine(a); // 5 (no cambia)

// Reference type: ambas apuntan al mismo objeto
var libro1 = new Libro { Titulo = "1984" };
var libro2 = libro1;
libro2.Titulo = "Otro título";
Console.WriteLine(libro1.Titulo); // "Otro título" (¡cambió también libro1!)
```

decimal merece mención aparte: en Java usarías `BigDecimal` para cálculos monetarios exactos (evitar errores de coma flotante). En C#, `decimal` es un tipo nativo del lenguaje pensado exactamente para eso — lo verás en cualquier campo de precio o importe en Espiral MS.

1.3 Propiedades — La diferencia más importante

En Java, para encapsular un campo usas getters/setters manuales:

```
public class Persona {
    private String nombre;
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
}
```

En C#, esto se simplifica con **propiedades**:

```
public class Persona
{
    public string Nombre { get; set; }
}
```

Uso:

```
var p = new Persona();
p.Nombre = "Ana";
Console.WriteLine(p.Nombre);
```

Propiedades con lógica personalizada

```
public class Persona
{
    private string _nombre = string.Empty;

    public string Nombre
    {
        get { return _nombre; }
        set { _nombre = value.Trim(); } // "value" es una palabra reservada aquí
    }
}
```

Propiedades de solo lectura y auto-inicializadas

```
public class Libro
{
    // Solo se puede asignar en el constructor, nunca después
    public string Isbn { get; init; } = string.Empty;

    // Solo lectura calculada, sin campo de respaldo
    public string TituloCompleto => $"{Titulo} ({Anio})";
}
```

init (C# 9+) es útil para propiedades que deben fijarse al crear el objeto y no cambiar después — más estricto que set pero más flexible que un constructor obligatorio.

1.4 Nulabilidad

Java moderno usa `Optional<T>` para expresar “esto puede no tener valor”. C# resuelve esto de forma distinta, con soporte nativo del lenguaje:

```
// Nullable value types: value types normalmente no pueden ser null
int? edad = null; // el "?" habilita null para un value type
if (edad.HasValue)
{
    Console.WriteLine(edad.Value);
}

// Nullable reference types (activado por defecto en proyectos nuevos)
string? nombre = null; // el compilador avisa si lo usas sin comprobar

// Operador de coalescencia nula (??): valor por defecto si es null
string nombreFinal = nombre ?? "Sin nombre";

// Operador de navegación segura (?.): evita NullReferenceException
int? longitud = nombre?.Length;

// Combinando ambos: cadena de accesos seguros con valor por defecto
string resultado = persona?.Direccion?.Ciudad ?? "Ciudad desconocida";
```

Por qué importa: en Java, un `NullPointerException` puede aparecer en cualquier punto sin aviso previo del compilador. En C# con *nullable reference types* activado, el compilador te avisa en tiempo de compilación de posibles nulos no comprobados — mucho antes de llegar a producción.

1.5 Convenciones de nombres

- Java: métodos y variables en camelCase, clases en PascalCase
- C#: **todo lo público** (clases, métodos, propiedades) en PascalCase. Variables locales y parámetros en camelCase. Campos privados con _ de prefijo.

```
public class GestorPedidos
{
    private int _totalPedidos;           // campo privado: _camelCase

    public int TotalPedidos { get; set; } // propiedad pública: PascalCase

    public void ProcesarPedido(int idPedido) // método: PascalCase, parámetro: camelCase
    {
        int cantidadProcesada = 0;       // variable local: camelCase
    }
}
```

1.6 Interfaces y herencia

```
public interface IRepository<T>
{
    T ObtenerPorId(int id);
}

public class RepositorioUsuarios : IRepository<Usuario>
{
    public Usuario ObtenerPorId(int id)
    {
        return new Usuario();
    }
}
```

Nota de nomenclatura: en C# las interfaces se prefijan con I por convención (IRepositorio, IEnumerable, IDisposable). En Java esto no es obligatorio, aunque algunos equipos lo adoptan igualmente.

Herencia de clases

```
public class Animal
{
    public string Nombre { get; set; } = string.Empty;
    public virtual void HacerSonido() => Console.WriteLine("...");
}

public class Perro : Animal // ":" en vez de "extends"
{
    public override void HacerSonido() => Console.WriteLine("Guau");
}
```

Diferencias con Java: extends → :, @Override → override (y el método base debe marcarse virtual explícitamente para poder sobrescribirse — en Java todos los métodos son sobrescribibles por defecto salvo final).

1.7 Control de flujo — pequeñas diferencias útiles

```
// switch expression (C# 8+), mucho más compacto que switch clásico
string categoria = anio switch
{
    < 1900 => "Clásico",
    >= 1900 and < 2000 => "Siglo XX",
    _ => "Contemporáneo"
};

// pattern matching con "is"
if (objeto is Libro libro && libro.Anio > 2000)
{
    Console.WriteLine(libro.Titulo);
}

// range operator para arrays/listas
int[] numeros = { 1, 2, 3, 4, 5 };
int[] mitad = numeros[1..3]; // elementos en posiciones 1 y 2
```

1.8 Ejercicios Lección 1

1. Crea una clase Libro con propiedades Título, Autor y Anio
2. Crea una interfaz IPrestable con un método Prestar()
3. Implementa la interfaz en Libro
4. Crea una lista de libros y recórrela con foreach
5. Añade una propiedad calculada EsAntiguo que devuelva true si Anio < 1950
6. Usa switch expression para clasificar libros por siglo (XIX, XX, XXI)
7. Practica ?. y ?? con una propiedad Autor que pueda ser null

Lección 2: Colecciones y LINQ

[↑ Volver al índice](#)

2.1 Colecciones principales

Java	C#	Uso
ArrayList<T> / List<T> HashMap<K,V> HashSet<T>	List<T> Dictionary<K,V> HashSet<T>	Lista dinámica, la más usada Clave-valor, acceso O(1) Elementos únicos, sin orden garantizado
LinkedList<T>	LinkedList<T>	Lista enlazada, inserciones rápidas en extremos
T[] Queue<T>	T[] Queue<T>	Array de tamaño fijo Cola FIFO (primero en entrar, primero en salir)

Java	C#	Uso
Stack<T> (Deque)	Stack<T>	Pila LIFO (último en entrar, primero en salir)
TreeMap<K,V>	SortedDictionary<K,V>	Diccionario ordenado por clave

Dictionary en profundidad

```
Dictionary<string, int> edades = new Dictionary<string, int>
{
    { "Ana", 25 },
    { "Luis", 30 }
};

edades["Ana"] = 26;           // actualizar
edades["Carlos"] = 40;       // añadir nueva clave

// Acceso seguro sin lanzar excepción si no existe la clave
if (edades.TryGetValue("Pedro", out int edadPedro))
{
    Console.WriteLine(edadPedro);
}
else
{
    Console.WriteLine("Pedro no está en el diccionario");
}

foreach (var par in edades)
{
    Console.WriteLine($"{par.Key} tiene {par.Value} años");
}

// Solo claves o solo valores
foreach (var clave in edades.Keys) { /* ... */ }
foreach (var valor in edades.Values) { /* ... */ }
```

Queue y Stack

```
// Queue: útil para colas de procesamiento (ej: tareas pendientes)
Queue<string> colaTareas = new();
colaTareas.Enqueue("Tarea 1");
colaTareas.Enqueue("Tarea 2");
string siguiente = colaTareas.Dequeue(); // "Tarea 1"

// Stack: útil para deshacer/rehacer, o recorridos en profundidad
Stack<string> historial = new();
historial.Push("Página A");
historial.Push("Página B");
string ultima = historial.Pop(); // "Página B"
```

2.2 LINQ — Streams pero mejor

En Java usarías:

```
List<String> nombres = personas.stream()
    .filter(p -> p.getEdad() > 18)
    .map(Persona::getNombre)
    .collect(Collectors.toList());
```

En C# tienes **dos sintaxis** para LINQ:

Sintaxis de método (la que usarás casi siempre)

```
var nombres = personas
    .Where(p => p.Edad > 18)
    .Select(p => p.Nombre)
    .ToList();
```

Sintaxis de consulta (parecida a SQL, poco usada en la práctica)

```
var nombres = (from p in personas
    where p.Edad > 18
    select p.Nombre).ToList();
```

La sintaxis de método es la estándar en código profesional de .NET — es la que verás en Espiral MS.

2.3 Métodos LINQ más usados

Método	Equivalente Java	Ejemplo
Where	.filter()	libros.Where(l => l.Anio > 2000)
Select	.map()	libros.Select(l => l.Titulo)
OrderBy / OrderByDescending	.sorted()	libros.OrderBy(l => l.Anio)
ThenBy	.thenComparing()	libros.OrderBy(l => l.Autor).ThenBy(l => l.Anio)
First / FirstOrDefault	.findFirst()	libros.FirstOrDefault(l => l.Autor == "Cervantes")
Single / SingleOrDefault	—	Espera exactamente 1 resultado, lanza excepción si hay más de uno
Any	.anyMatch()	libros.Any(l => l.Anio < 1700)
All	.allMatch()	libros.All(l => l.Autor != null)
Count	.count()	libros.Count(l => l.Anio > 1900)
Sum / Average / Max / Min	.sum() / .average()	libros.Sum(l => l.Anio)

Método	Equivalente Java	Ejemplo
GroupBy	.collect(groupingBy())	libros.GroupBy(l => l.Autor)
Distinct	.distinct()	libros.Select(l => l.Autor).Distinct()
Skip / Take	.skip() / .limit()	libros.Skip(10).Take(5) (paginación)
ToList / ToArray / ToDictionary	.collect(toList())	.ToList()

2.4 Encadenando operaciones

```
var resultado = libros
    .Where(l => l.Anio > 1900)
    .OrderByDescending(l => l.Anio)
    .Select(l => l.Titulo)
    .ToList();
```

Paginación con Skip + Take (patrón muy común en APIs)

```
int pagina = 2;
int tamanoPagina = 10;

var librosPagina = libros
    .OrderBy(l => l.Titulo)
    .Skip((pagina - 1) * tamanoPagina)
    .Take(tamanoPagina)
    .ToList();
```

Agregaciones combinadas

```
var estadisticas = new
{
    Total = libros.Count,
    AnioMasAntiguo = libros.Min(l => l.Anio),
    AnioMasReciente = libros.Max(l => l.Anio),
    PromedioAnio = libros.Average(l => l.Anio)
};
```

2.5 Ejecución diferida (deferred execution)

Esto es crucial y suele confundir a quien viene de Java: la mayoría de operadores LINQ (Where, Select, OrderBy) son *perezosos*. No se ejecutan hasta que alguien recorre el resultado.

```
var libros2020 = libros.Where(l => l.Anio >= 2020); // NO filtra todavía, solo define "la re
libros.Add(new Libro { Anio = 2021 }); // si añades algo antes de iterar, SÍ se incluirá
```

```
foreach (var libro in libros2020) // aquí se ejecuta el filtro realmente
{
    Console.WriteLine(libro.Titulo);
}
```

Para forzar la ejecución inmediata (y “congelar” el resultado en ese momento), usa `.ToList()`, `.ToArray()` o `.Count()`:

```
var libros2020 = libros.Where(l => l.Anio >= 2020).ToList(); // se ejecuta AHORA, resultado
```

Por qué importa con Entity Framework (Lección 8): cuando trabajes con base de datos, la ejecución diferida significa que la consulta SQL real no se lanza hasta que iteras el resultado — puedes seguir añadiendo `.Where()` encima sin penalización, y EF Core combina todo en una sola consulta SQL optimizada.

2.6 Ejercicios Lección 2

Con una lista de al menos 5 libros:

1. Usa `Where` para obtener libros después de 1900
2. Usa `OrderBy` + `Select` para títulos ordenados por año
3. Usa `Any` para comprobar si existe algún libro sin autor
4. Usa `GroupBy` para agrupar por década (pista: `l.Anio / 10 * 10`)
5. Encadena `Where` + `OrderByDescending` + `Select` + `ToList`
6. Implementa paginación con `Skip` + `Take` (página de 2 elementos)
7. Calcula estadísticas: total, año más antiguo, año más reciente, promedio
8. Usa un `Dictionary<string, List<Libro>>` para agrupar libros por autor manualmente (sin `GroupBy`) y compara con la versión LINQ

Lección 3: Async/Await y manejo de excepciones

[↑ Volver al índice](#)

3.1 Manejo de excepciones

```
try
{
    int resultado = 10 / int.Parse("0");
}
catch (DivideByZeroException ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
catch (FormatException ex)
{
    Console.WriteLine($"Formato inválido: {ex.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"Error general: {ex.Message}");
}
```

```
finally
{
    Console.WriteLine("Siempre se ejecuta");
}
```

Diferencias con Java: - No existen las “checked exceptions” — ningún método te obliga a declarar throws - No hay obligación de declarar excepciones en la firma del método - throw new IllegalArgumentException("msg") → throw new ArgumentException("msg") - El orden de los catch importa: de más específico a más genérico (igual que en Java)

Relanzar excepciones correctamente

```
try
{
    ProcesarLibro();
}
catch (Exception ex)
{
    LoguearError(ex);
    throw;           // CORRECTO: conserva el stack trace original
    // throw ex;     // INCORRECTO: reinicia el stack trace, pierdes info de dónde falló
}
```

Excepciones personalizadas

```
public class LibroNoEncontradoException : Exception
{
    public LibroNoEncontradoException(string mensaje) : base(mensaje) { }
}

// Con datos adicionales
public class StockInsuficienteException : Exception
{
    public int StockDisponible { get; }

    public StockInsuficienteException(string mensaje, int stockDisponible) : base(mensaje)
    {
        StockDisponible = stockDisponible;
    }
}

// Uso
throw new StockInsuficienteException("No hay suficiente stock", stockDisponible: 2);
```

using — gestión automática de recursos

Equivalente a try-with-resources de Java:

```
// Java
try (FileReader fr = new FileReader("archivo.txt")) {
    // usar fr
}
```

```
// C# clásico
using (var fr = new StreamReader("archivo.txt"))
{
    // usar fr
} // se libera automáticamente al salir del bloque

// C# moderno (C# 8+), sin llaves, se libera al final del método/bloque contenedor
using var fr = new StreamReader("archivo.txt");
```

3.2 async/await

```
public async Task<string> ObtenerDatosAsync()
{
    Console.WriteLine("Empezando...");
    await Task.Delay(2000); // simula una operación que tarda (ej: llamada a API)
    Console.WriteLine("Terminado");
    return "Datos obtenidos";
}

public static async Task Main()
{
    string resultado = await ObtenerDatosAsync();
    Console.WriteLine(resultado);
}
```

Reglas clave: - Un método que usa await debe ser async - Su retorno es Task (sin valor) o Task<T> (con valor de tipo T) - await pausa la ejecución del método **sin bloquear el hilo** — el hilo queda libre para atender otras peticiones mientras espera - Por convención, métodos async terminan en Async (ObtenerDatosAsync, GuardarAsync)

¿Qué es Task<T> exactamente?

Task<T> representa una operación en curso que eventualmente producirá un valor de tipo T. Es como un resguardo de tintorería: el resguardo (Task) es la promesa de que la ropa (T) estará lista; await es ir a recogerla cuando esté.

Ejecutar tareas en paralelo con Task.WhenAll

```
public async Task ProcesarVariosLibrosAsync()
{
    Task<Libro> tarea1 = ObtenerLibroAsync(1);
    Task<Libro> tarea2 = ObtenerLibroAsync(2);
    Task<Libro> tarea3 = ObtenerLibroAsync(3);

    // Las 3 operaciones se ejecutan EN PARALELO, no una tras otra
    Libro[] libros = await Task.WhenAll(tarea1, tarea2, tarea3);

    foreach (var libro in libros)
        Console.WriteLine(libro.Titulo);
}
```

Si hicieras await de cada una por separado (await tarea1; await tarea2; await tarea3;), se ejecutarían **secuencialmente**, tardando la suma de los tres tiempos en vez del máximo de ellos.

Manejo de excepciones en código async

```
public async Task<string> LlamarApiAsync()
{
    try
    {
        var resultado = await AlgunaOperacionAsync();
        return resultado;
    }
    catch (HttpRequestException ex)
    {
        Console.WriteLine($"Fallo de red: {ex.Message}");
        throw; // relanza conservando el stack trace
    }
}
```

CancellationToken — cancelar operaciones en curso

En aplicaciones reales (APIs web, por ejemplo), necesitas poder cancelar una operación async si el cliente cierra la conexión o pasa un timeout:

```
public async Task<Libro> ObtenerLibroAsync(int id, CancellationToken token)
{
    token.ThrowIfCancellationRequested(); // comprobar si ya se pidió cancelar
    await Task.Delay(2000, token);        // Task.Delay también respeta el token
    return new Libro { Id = id };
}

// Uso con timeout de 5 segundos
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
try
{
    var libro = await ObtenerLibroAsync(1, cts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("La operación se canceló por timeout");
}
```

3.3 Por qué importa esto para la empresa

En Espiral MS, cuando accedas a base de datos con Entity Framework (Lección 8), verás patrones como este constantemente:

```
public async Task<List<Cliente>> ObtenerClientesAsync()
{
    return await _contexto.Clientes
        .Where(c => c.Activo)
        .ToListAsync(); // ← el mismo LINQ que ya conoces, con Async + await
}
```

Es exactamente el LINQ de la Lección 2, combinado con lo que acabas de aprender aquí.

3.4 Ejercicios Lección 3

1. Crea `DividirAsync(int a, int b)` con `Task.Delay(1000)`
 2. Llama desde Main con try/catch capturando `DivideByZeroException`
 3. Prueba con valores válidos y con `b = 0`
 4. Añade un finally que imprima "Operación finalizada"
 5. Crea 3 métodos async independientes (por ejemplo, simulando 3 llamadas a servicios distintos) y ejecútalos en paralelo con `Task.WhenAll`, comparando el tiempo total con hacerlo secuencial
 6. Crea una excepción personalizada `LibroNoDisponibleException` con una propiedad adicional `FechaDisponible`
 7. Practica using var con un `StreamWriter` para escribir texto a un archivo
-

Mini-proyecto: Gestor de Biblioteca

[↑ Volver al índice](#)

Estructura

```
GestorBiblioteca/  
├── Program.cs  
├── modelo/  
│   ├── Libro.cs  
├── excepciones/  
│   ├── LibroNoEncontradoException.cs  
│   └── LibroYaPrestadoException.cs  
└── repositorio/  
    └── BibliotecaRepositorio.cs
```

Modelo: Libro.cs

```
namespace modelo  
{  
    public class Libro  
    {  
        public int Id { get; set; }  
        public string Titulo { get; set; } = string.Empty;  
        public string? Autor { get; set; }  
        public int Anio { get; set; }  
        public bool Prestado { get; set; } = false;  
  
        public Libro(int id, string titulo, string? autor, int anio, bool prestado)  
        {  
            Id = id;  
            Titulo = titulo;  
            Autor = autor;  
            Anio = anio;  
            Prestado = prestado;  
        }  
    }  
}
```

```

    }

    public Libro() { }

    public event Action<string>? LibroPrestado;

    public void NotificarPrestamo(string mensaje)
    {
        LibroPrestado?.Invoke(mensaje);
    }
}

```

Excepciones

```

// excepciones/LibroNoEncontradoException.cs
namespace excepciones
{
    public class LibroNoEncontradoException : Exception
    {
        public LibroNoEncontradoException(string mensaje) : base(mensaje) { }
    }
}

// excepciones/LibroYaPrestadoException.cs
namespace excepciones
{
    public class LibroYaPrestadoException : Exception
    {
        public LibroYaPrestadoException(string mensaje) : base(mensaje) { }
    }
}

```

Repositorio

```

using modelo;
using excepciones;

namespace repositorio
{
    public class BibliotecaRepositorio
    {
        private List<Libro> _libros = new List<Libro>();

        public BibliotecaRepositorio()
        {
            _libros.Add(new Libro(1, "Cien años de soledad", "Gabriel García Márquez", 1967, false));
            _libros.Add(new Libro(2, "1984", "George Orwell", 1949, false));
            _libros.Add(new Libro(3, "El Principito", "Antoine de Saint-Exupéry", 1943, false));
            _libros.Add(new Libro(4, "Don Quijote", "Miguel de Cervantes", 1605, false));
            _libros.Add(new Libro(5, "Moby Dick", "Herman Melville", 1851, false));
            _libros.Add(new Libro(6, "La Odisea", "Homero", -800, false));
        }
    }
}

```

```

public List<Libro> ObtenerTodos() => _libros;

public List<Libro> BuscarPorTitulo(string texto)
    => _libros.Where(l => l.Titulo.Contains(texto, StringComparison.OrdinalIgnoreCase));

public Libro ObtenerPorId(int id)
    => _libros.FirstOrDefault(l => l.Id == id)
    ?? throw new LibroNoEncontradoException($"Libro con ID {id} no encontrado.");

public async Task PrestarAsync(int id)
{
    var libro = ObtenerPorId(id);
    if (libro.Prestado)
        throw new LibroYaPrestadoException($"El libro '{libro.Titulo}' ya está prestado.");

    await Task.Delay(500);
    libro.Prestado = true;
    libro.NotificarPrestamo($"El libro '{libro.Titulo}' ha sido prestado.");
}

public async Task DevolverAsync(int id)
{
    var libro = ObtenerPorId(id);
    if (!libro.Prestado)
        throw new LibroYaPrestadoException($"El libro '{libro.Titulo}' no está prestado.");

    await Task.Delay(500);
    libro.Prestado = false;
}

public List<Libro> ObtenerDisponibles()
    => _libros.Where(l => !l.Prestado).ToList();

public List<Libro> ObtenerOrdenadosPorAnio()
    => _libros.OrderBy(l => l.Anio).ToList();
}
}

```

Lección 4: Generics

[↑ Volver al índice](#)

4.1 Generics “reificados”

En Java usan *type erasure*: en tiempo de ejecución, `List<String>` y `List<Integer>` son indistinguibles (ambas son solo `List`). En C# los generics son **reificados** — se mantiene la información del tipo en runtime:

```

public static void MostrarTipo<T>(T valor)
{
    Console.WriteLine($"El tipo es: {typeof(T).Name}");
}

```

```
MostrarTipo(42);           // "El tipo es: Int32"
MostrarTipo("hola");       // "El tipo es: String"
```

Esto tiene una consecuencia práctica importante: en C#, `List<int>` no necesita autoboxing de cada elemento (a diferencia de Java, donde `List<Integer>` convierte cada `int` primitivo en un objeto `Integer`). Resultado: mejor rendimiento con tipos value type en colecciones genéricas.

4.2 Clase genérica

```
public class Contenedor<T>
{
    private T _valor;

    public Contenedor(T valor)
    {
        _valor = valor;
    }

    public T ObtenerValor() => _valor;

    public void MostrarTipo()
    {
        Console.WriteLine($"Tipo: {typeof(T).Name}");
    }
}

// Uso
var contenedorTexto = new Contenedor<string>("Hola");
var contenedorNumero = new Contenedor<int>(42); // en Java sería Contenedor<Integer>
```

Método genérico independiente

```
public static T ObtenerPrimero<T>(List<T> lista)
{
    if (lista.Count == 0)
        throw new InvalidOperationException("La lista está vacía");
    return lista[0];
}

// El compilador infiere T automáticamente por el tipo del argumento
Libro primero = ObtenerPrimero(misLibros); // T = Libro, sin especificarlo explícitamente
```

4.3 Constraints

```
public class Repositorio<T> where T : class, new()
{
    public T CrearNuevo() => new T();
}
```

Constraints comunes:

Constraint	Significado
where T : class	T debe ser un tipo referencia
where T : struct	T debe ser un value type
where T : new()	T debe tener constructor público sin parámetros
where T : IComparable<T>	T debe implementar esa interfaz
where T : Libro	T debe ser Libro o una clase derivada
where T : class, IEntity, new()	Se pueden combinar varios constraints

Ejemplo aplicado — repositorio genérico reutilizable

```
public interface IEntity
{
    int Id { get; set; }
}

public class RepositorioGenerico<T> where T : IEntity
{
    private List<T> _elementos = new();

    public void Agregar(T elemento) => _elementos.Add(elemento);

    public T ObtenerPorId(int id)
    {
        return _elementos.FirstOrDefault(e => e.Id == id)
            ?? throw new InvalidOperationException($"Elemento {id} no encontrado.");
    }

    public List<T> ObtenerTodos() => new List<T>(_elementos);
}

// Uso: la misma clase sirve para Libro, Cliente, Pedido...
public class Libro : IEntity
{
    public int Id { get; set; }
    public string Titulo { get; set; } = string.Empty;
}

var repo = new RepositorioGenerico<Libro>();
```

Este patrón es exactamente la base sobre la que funciona DbSet<T> en Entity Framework (Lección 8) — un repositorio genérico que sabe operar con cualquier entidad.

4.4 Múltiples parámetros de tipo

```
public class Par<TClave, TValor>
{
    public TClave Clave { get; set; } = default!;
    public TValor Valor { get; set; } = default!;
}
```

```
var par = new Par<string, int> { Clave = "edad", Valor = 30 };
```

Esto es literalmente cómo está construido Dictionary<TKey, TValue> internamente.

4.5 Ejercicios Lección 4

1. Crea una interfaz IEntidad con propiedad Id
2. Haz que Libro implemente IEntidad
3. Crea RepositorioGenerico<T> where T : IEntidad
4. Implementa Agregar(T), ObtenerPorId(int), ObtenerTodos()
5. Instancia con RepositorioGenerico<Libro>
6. Añade un método genérico Filtrar<T>(List<T> lista, Func<T, bool> criterio) y pruébalo con distintos tipos
7. Crea una clase Par<TClave, TValor> y úsala para representar una estadística (ej: Par<string, int> para "autor" → "cantidad de libros")

Lección 5: Delegates y Eventos

[↑ Volver al índice](#)

5.1 ¿Qué es un delegate?

Un **delegate** es un tipo que representa una referencia a un método. En Java, lo más parecido son las interfaces funcionales (Function<T,R>, Predicate<T>) combinadas con lambdas.

```
public delegate bool Criterio(Libro libro);

public static bool EsAntiguo(Libro libro) => libro.Anio < 1950;

Criterio delegado = EsAntiguo;
bool resultado = delegado(unLibro); // llama internamente a EsAntiguo
```

5.2 Func, Action y Predicate — predefinidos

En la práctica, casi nunca se crean delegates personalizados. Se usan los genéricos predefinidos del framework:

Delegate	Firma	Ejemplo
Action	nada → nada	Action saludar = () => Console.WriteLine("Hola");
Action<T>	T → nada	Action<string> imprimir = s => Console.WriteLine(s);
Func<T, TResult>	T → TResult	Func<int, int> cuadrado = x => x * x;
Func<T1, T2, TResult>	T1, T2 → TResult	Func<int, int, int> sumar = (a, b) => a + b;

Delegate	Firma	Ejemplo
Predicate<T>	T → bool	Predicate<Libro> antiguo = l => l.Anio < 1950;

La conexión con LINQ

```
public static List<T> Filtrar<T>(List<T> lista, Func<T, bool> criterio)
{
    return lista.Where(criterio).ToList();
}
```

Func<T, bool> es exactamente lo que recibe .Where() por debajo — cuando escribes libros.Where(l => l.Anio > 2000), esa lambda ES un Func<Libro, bool>.

5.3 Delegates multicast

Un delegate puede apuntar a **varios métodos a la vez**, ejecutándose todos en orden al invocarlo:

```
Action miAccion = () => Console.WriteLine("Primero");
miAccion += () => Console.WriteLine("Segundo");
miAccion += () => Console.WriteLine("Tercero");

miAccion(); // ejecuta los tres, en el orden en que se añadieron
```

Esto es la base técnica sobre la que funcionan los **eventos**.

5.4 Eventos

Un evento es un delegate especializado para el patrón publicador/suscriptor (Observer):

```
public class GestorNotificaciones
{
    public event Action<string>? NotificacionEnviada;

    public void EnviarNotificacion(string mensaje)
    {
        NotificacionEnviada?.Invoke(mensaje);
    }
}

// Uso
var gestor = new GestorNotificaciones();
gestor.NotificacionEnviada += msg => Console.WriteLine($"Notificación: {msg}");
gestor.EnviarNotificacion("Hola"); // Dispara el evento
```

¿Por qué ?.Invoke? Si nadie se ha suscrito todavía, el evento es null. El operador ?. evita un NullReferenceException en ese caso.

Diferencia entre un delegate público normal y un event

```
public class Ejemplo
{
    public Action<string>? DelegatePublico; // cualquiera puede hacer .Invoke() desde fuera
    public event Action<string>? EventoReal; // solo la propia clase puede invocarlo
}
```

Declarar con event restringe: código externo solo puede suscribirse (+) o desuscribirse (-), nunca disparar el evento directamente. Es una protección de encapsulación que un delegate público normal no ofrece.

Desuscribirse de un evento

```
void ManejarNotificacion(string mensaje) => Console.WriteLine(mensaje);

gestor.NotificacionEnviada += ManejarNotificacion;
// ...más tarde...
gestor.NotificacionEnviada -= ManejarNotificacion; // deja de recibir notificaciones
```

Nota: solo puedes desuscribir con -= si guardaste una referencia nombrada al método — una lambda anónima (msg => ...) no se puede desuscribir después porque no tienes forma de referenciarla de nuevo.

5.5 En el mundo empresarial

- **Func/Action/Predicate:** constantemente en LINQ, callbacks, configuración de servicios y middlewares
- **Eventos:** comunes en interfaces gráficas (WinForms, WPF, Blazor) y en arquitecturas orientadas a eventos en backend (ej: “cuando se crea un pedido, notificar por email”)

5.6 Ejercicios Lección 5

1. Crea GestorNotificaciones con evento NotificacionEnviada
2. Añade método EnviarNotificacion(string mensaje)
3. Suscribe 3 lambdas diferentes
4. Envía una notificación y verifica que se ejecutan todas
5. Aplica esto a Libro: evento LibroPrestado
6. Crea un método con nombre (no lambda) y practica suscribir/desuscribir con += / -=
7. Diferencia en código un delegate público normal de un event, e intenta invocar el evento desde fuera de la clase (comprueba que el compilador no te deja)

Lección 6: Patrones de Diseño

[↑ Volver al índice](#)

6.1 ¿Por qué patrones de diseño?

Un patrón de diseño es una solución probada a un problema recurrente. No es código específico, es una **estructura reutilizable**. Beneficios: código más legible para el resto del equipo, más fácil

de testear, y más fácil de modificar sin romper otras partes.

6.2 Repository Pattern

Abstrae el acceso a datos detrás de una interfaz, para que el resto del código no sepa (ni le importe) si los datos vienen de memoria, un archivo o una base de datos.

```
public interface ILibroRepositorio
{
    List<Libro> ObtenerTodos();
    Libro ObtenerPorId(int id);
    Task PrestarAsync(int id);
}

public class BibliotecaRepositorio : ILibroRepositorio
{
    // Implementación (en memoria, en tu proyecto actual)
}

// Uso: el código consumidor depende de la interfaz, no de la implementación concreta
ILibroRepositorio repo = new BibliotecaRepositorio();
```

Ventaja real: si mañana cambias de “datos en memoria” a “SQL Server con Entity Framework”, solo creas BibliotecaRepositorioEF : ILibroRepositorio y cambias una línea en Program.cs. Ningún otro archivo se entera del cambio.

6.3 Dependency Injection (DI)

Pasar a un objeto lo que necesita (sus dependencias) por constructor, en vez de que él mismo las cree.

Mal (acoplado)

```
public class GestorPrestamos
{
    private BibliotecaRepositorio _repo = new(); // crea su propia dependencia, acoplamiento
    public async Task Prestar(int id)
    {
        await _repo.PrestarAsync(id);
    }
}
```

Bien (inyectado)

```
public class GestorPrestamos
{
    private readonly ILibroRepositorio _repo; // depende de la interfaz, no de una clase concreta
    public GestorPrestamos(ILibroRepositorio repo) // inyección por constructor
    {
    }
```

```

        _repo = repo;
    }

    public async Task Prestar(int id)
    {
        await _repo.PrestarAsync(id);
    }
}

// Uso
ILibroRepositorio repo = new BibliotecaRepositorio();
var gestor = new GestorPrestamos(repo);

```

Ahora GestorPrestamos funciona con **cualquier** implementación de ILibroRepositorio — incluida una falsa para tests.

Los 3 “lifetimes” de DI en ASP.NET Core (lo que verás en las prácticas)

Cuando llegues a ASP.NET Core, registrarás las dependencias con `builder.Services.Add...`, y elegirás uno de estos tres ciclos de vida:

Lifetime	Comportamiento	Cuándo usarlo
AddTransient<T>	Nueva instancia cada vez que se solicita	Servicios ligeros, sin estado
AddScoped<T>	Una instancia por petición HTTP	El más común — ej: DbContext
AddSingleton<T>	Una única instancia para toda la app	Configuración, caché compartida

```

// Ejemplo de cómo se verá en Program.cs de un proyecto ASP.NET Core real
builder.Services.AddScoped<ILibroRepositorio, BibliotecaRepositorio>();
builder.Services.AddScoped<GestorPrestamos>();

```

Esto automatiza exactamente lo que hicimos a mano arriba: cuando algo pida un ILibroRepositorio, el framework crea automáticamente un BibliotecaRepositorio y lo inyecta.

6.4 Factory Pattern

Una factory es un método (o clase) que crea objetos por ti, encapsulando la lógica de construcción cuando esta es compleja.

```

public static class BibliotecaFactory
{
    public static GestorPrestamos CrearGestor()
    {
        var repo = new BibliotecaRepositorio();
        return new GestorPrestamos(repo);
    }
}

// Uso
var gestor = BibliotecaFactory.CrearGestor();

```

En ASP.NET Core moderno, esto se automatiza con el contenedor de DI (visto arriba), pero el concepto de “encapsular la construcción compleja de un objeto” sigue siendo el mismo.

6.5 Singleton Pattern

Garantiza que solo existe **una instancia** de una clase en toda la aplicación.

```
public class Configuracion
{
    public static Configuracion Instancia { get; } = new();

    private Configuracion() { } // constructor privado: nadie puede hacer "new" desde fuera

    public string ObtenerRutaBaseDatos() => "Server=...";
}

// Uso
var config = Configuracion.Instancia; // siempre el mismo objeto
```

Cuidado: los singletons manuales pueden complicar los tests unitarios (estado compartido global). En ASP.NET Core, se prefiere `AddSingleton<T>` con DI, que resuelve este problema porque el propio framework gestiona el ciclo de vida y puede sustituirlo por un mock en tests.

6.6 Mock para testing

```
public class RepositorioMock : ILibroRepositorio
{
    private List<Libro> _libros = new()
    {
        new Libro(1, "Test Book", "Test Author", 2020, false)
    };

    public List<Libro> ObtenerTodos() => _libros;
    public Libro ObtenerPorId(int id) => _libros.First(l => l.Id == id);
    public async Task PrestarAsync(int id)
    {
        await Task.Delay(10); // sin latencia real, tests rápidos
        _libros.First(l => l.Id == id).Prestado = true;
    }
}

// En un test
ILibroRepositorio repo = new RepositorioMock();
var gestor = new GestorPrestamos(repo);
await gestor.Prestar(1);
// se puede comprobar el resultado sin tocar una base de datos real
```

Esto es lo que hace posible testear `GestorPrestamos` sin necesitar una base de datos de verdad — el corazón de por qué `Repository` + `DI` se usan juntos casi siempre.

6.7 Cómo se combinan estos patrones en la práctica

Program.cs (composición)

```
|
|├ registra: ILibroRepositorio → BibliotecaRepositorio (Repository)
|├ registra: GestorPrestamos (recibe DI automática)
|
└ en tests: ILibroRepositorio → RepositorioMock (mismo contrato, otra implementación)
```

En Espiral MS verás esta combinación constantemente: interfaz + implementación real + implementación de test, todas intercambiables gracias a DI.

6.8 Ejercicios Lección 6

1. Crea ILibroRepositorio con todos los métodos
2. Haz que BibliotecaRepositorio lo implemente
3. Crea GestorPrestamos con inyección de DI
4. Crea RepositorioMock para testing
5. Refactoriza Program.cs para usar patrones
6. Crea una BibliotecaFactory que construya un GestorPrestamos completamente configurado
7. Implementa un Configuracion como Singleton con al menos 2 propiedades (ej: nombre de la app, versión)
8. Escribe un pequeño programa que use RepositorioMock en vez de BibliotecaRepositorio y verifica que GestorPrestamos funciona igual con ambos

Lección 7: SQL — Oracle y T-SQL

[↑ Volver al índice](#)

7.1 DDL — Crear la estructura de la base de datos

```
CREATE TABLE Clientes (
  IdCliente INT PRIMARY KEY IDENTITY(1,1), -- IDENTITY = auto-incremento (T-SQL)
  NombreCliente NVARCHAR(100) NOT NULL,
  Email NVARCHAR(100),
  Pais NVARCHAR(50) NOT NULL,
  Activo BIT NOT NULL DEFAULT 1,
  FechaRegistro DATETIME NOT NULL DEFAULT GETDATE()
);

CREATE TABLE Pedidos (
  IdPedido INT PRIMARY KEY IDENTITY(1,1),
  IdCliente INT NOT NULL,
  FechaPedido DATETIME NOT NULL DEFAULT GETDATE(),
  Total DECIMAL(10, 2) NOT NULL DEFAULT 0,
  FOREIGN KEY (IdCliente) REFERENCES Clientes(IdCliente)
);

-- Modificar una tabla existente
ALTER TABLE Clientes ADD Telefono NVARCHAR(20);
```

```
ALTER TABLE Clientes DROP COLUMN Telefono;
```

```
-- Eliminar una tabla  
DROP TABLE Pedidos;
```

En Oracle, el equivalente de IDENTITY es una SEQUENCE combinada con un TRIGGER, o directamente GENERATED ALWAYS AS IDENTITY en versiones modernas (Oracle 12c+).

7.2 DML — Manipular datos

```
-- INSERT  
INSERT INTO Clientes (NombreCliente, Email, Pais, Activo)  
VALUES ('Ana García', 'ana@example.com', 'España', 1);  
  
-- UPDATE  
UPDATE Clientes  
SET Activo = 0  
WHERE IdCliente = 5;  
  
-- DELETE  
DELETE FROM Clientes WHERE Activo = 0 AND FechaRegistro < '2020-01-01';
```

7.3 Consultas básicas

```
SELECT * FROM Clientes WHERE Activo = 1 ORDER BY Nombre;  
  
SELECT Nombre, Email FROM Clientes WHERE Pais = 'España';  
  
SELECT COUNT(*) AS Total FROM Clientes;  
  
-- Combinando condiciones  
SELECT * FROM Clientes  
WHERE Pais = 'España' AND Activo = 1 AND FechaRegistro >= '2025-01-01';  
  
-- LIKE para búsquedas parciales  
SELECT * FROM Clientes WHERE NombreCliente LIKE 'Ana%'; -- empieza por "Ana"  
SELECT * FROM Clientes WHERE Email LIKE '%@gmail.com'; -- termina en "@gmail.com"  
  
-- IN para listas de valores  
SELECT * FROM Clientes WHERE Pais IN ('España', 'México', 'Argentina');  
  
-- BETWEEN para rangos  
SELECT * FROM Pedidos WHERE Total BETWEEN 50 AND 200;
```

7.4 Joins — los 4 tipos principales

```
-- INNER JOIN: solo filas que coinciden en ambas tablas  
SELECT c.NombreCliente, p.Total
```

```

FROM Clientes c
INNER JOIN Pedidos p ON c.IdCliente = p.IdCliente;

-- LEFT JOIN: todas las filas de la izquierda, coincidan o no
SELECT c.NombreCliente, COUNT(p.IdPedido) AS TotalPedidos
FROM Clientes c
LEFT JOIN Pedidos p ON c.IdCliente = p.IdCliente
GROUP BY c.IdCliente, c.NombreCliente;
-- (clientes sin pedidos aparecen igual, con TotalPedidos = 0)

-- RIGHT JOIN: todas las filas de la derecha, coincidan o no (menos usado)
SELECT c.NombreCliente, p.Total
FROM Clientes c
RIGHT JOIN Pedidos p ON c.IdCliente = p.IdCliente;

-- FULL OUTER JOIN: todas las filas de ambas tablas
SELECT c.NombreCliente, p.Total
FROM Clientes c
FULL OUTER JOIN Pedidos p ON c.IdCliente = p.IdCliente;

```

Join múltiple (3+ tablas), el más común en consultas reales

```

SELECT c.NombreCliente, pr.Nombre AS Producto, dp.Cantidad
FROM Clientes c
INNER JOIN Pedidos pe ON c.IdCliente = pe.IdCliente
INNER JOIN DetallesPedido dp ON pe.IdPedido = dp.IdPedido
INNER JOIN Productos pr ON dp.IdProducto = pr.IdProducto
WHERE c.Activo = 1;

```

7.5 GroupBy y agregaciones

```

SELECT Pais, COUNT(*) AS Total
FROM Clientes
GROUP BY Pais
HAVING COUNT(*) > 5 -- HAVING filtra DESPUÉS de agrupar (WHERE filtra ANTES)
ORDER BY Total DESC;

-- Varias agregaciones a la vez
SELECT
    Pais,
    COUNT(*) AS TotalClientes,
    AVG(TotalGastado) AS GastoPromedio,
    MAX(TotalGastado) AS GastoMaximo,
    MIN(FechaRegistro) AS PrimerRegistro
FROM Clientes
GROUP BY Pais;

```

Diferencia clave WHERE vs HAVING: WHERE filtra filas individuales antes de agrupar; HAVING filtra grupos ya formados. Por eso HAVING COUNT(*) > 5 funciona pero WHERE COUNT(*) > 5 da error.

7.6 Subconsultas

```
-- Subconsulta en el WHERE
SELECT NombreCliente
FROM Clientes
WHERE IdCliente IN (
    SELECT IdCliente FROM Pedidos WHERE Total > 100
);

-- Subconsulta correlacionada (se ejecuta por cada fila externa)
SELECT c.NombreCliente,
       (SELECT COUNT(*) FROM Pedidos p WHERE p.IdCliente = c.IdCliente) AS TotalPedidos
FROM Clientes c;

-- Common Table Expression (CTE) – más legible que subconsultas anidadas
WITH ClientesActivos AS (
    SELECT * FROM Clientes WHERE Activo = 1
)
SELECT NombreCliente, Pais FROM ClientesActivos WHERE Pais = 'España';
```

7.7 Índices — por qué importan para el rendimiento

```
-- Crear un índice sobre una columna consultada frecuentemente
CREATE INDEX IX_Clientes_Email ON Clientes(Email);

-- Índice único (además de optimizar, impide duplicados)
CREATE UNIQUE INDEX UX_Clientes_Email ON Clientes(Email);

-- Índice compuesto (varias columnas, útil si sueles filtrar por ambas juntas)
CREATE INDEX IX_Pedidos_Cliente_Fecha ON Pedidos(IdCliente, FechaPedido);
```

Sin índice, `WHERE Email = '...'` recorre toda la tabla fila por fila (*table scan*). Con índice, la búsqueda es casi instantánea incluso con millones de filas. La contrapartida: cada índice ralentiza ligeramente los `INSERT/UPDATE`, así que no se indexa todo indiscriminadamente.

7.8 Procedimientos almacenados

Oracle

```
CREATE PROCEDURE sp_ObtenerClientes(
    p_Pais VARCHAR2,
    p_Resultado OUT SYS_REFCURSOR
)
IS
BEGIN
    OPEN p_Resultado FOR
        SELECT IdCliente, Nombre, Email
        FROM Clientes
        WHERE Pais = p_Pais AND Activo = 1;
END;
```

SQL Server (T-SQL)

```
CREATE PROCEDURE sp_InsertarCliente
    @Nombre NVARCHAR(100),
    @Email NVARCHAR(100),
    @Pais NVARCHAR(50),
    @IdNuevo INT OUTPUT
AS
BEGIN
    INSERT INTO Clientes (NombreCliente, Email, Pais, Activo)
    VALUES (@Nombre, @Email, @Pais, 1);

    SET @IdNuevo = @@IDENTITY;
END;

-- Ejecutar el procedimiento
DECLARE @NuevoId INT;
EXEC sp_InsertarCliente 'Ana García', 'ana@example.com', 'España', @NuevoId OUTPUT;
SELECT @NuevoId;
```

Por qué se usan: centralizan lógica de acceso a datos en la propia BD, reducen el tráfico de red (una sola llamada en vez de varias consultas), y en muchas empresas siguen siendo el estándar para operaciones críticas.

7.9 Diferencias prácticas Oracle vs SQL Server

Aspecto	Oracle	SQL Server (T-SQL)
Auto-incremento	SEQUENCE + TRIGGER, o GENERATED ALWAYS AS IDENTITY	IDENTITY(1,1)
Limitar filas	WHERE ROWNUM <= 10 (o FETCH FIRST 10 ROWS ONLY)	SELECT TOP 10
Concatenar texto	\\ \\	+ o CONCAT()
Fecha actual	SYSDATE	GETDATE()
Booleano	No existe nativo, se usa NUMBER(1)	BIT
Cadena de texto	VARCHAR2	NVARCHAR

7.10 Ejercicios Lección 7

1. Crea las tablas Clientes y Pedidos con sus relaciones (FK)
2. Inserta al menos 5 clientes y 8 pedidos
3. Escribe una consulta que obtenga clientes de “España” con más de 1 pedido (usando HAVING)
4. Escribe un LEFT JOIN que muestre todos los clientes, incluso los que no tienen pedidos
5. Crea un índice sobre la columna Email de Clientes
6. Escribe una subconsulta que obtenga el nombre de los clientes cuyo gasto total supere el promedio de todos los clientes
7. Escribe un procedimiento almacenado que reciba un país y devuelva el número de clientes activos en ese país

Lección 8: Entity Framework Core

[↑ Volver al índice](#)

8.1 ¿Qué es EF Core?

Un ORM que traduce LINQ a SQL automáticamente:

```
// Escribes esto
var activos = await _contexto.Clientes
    .Where(c => c.Activo)
    .ToListAsync();

// EF Core genera:
// SELECT * FROM Clientes WHERE Activo = 1;
```

8.2 Instalación de paquetes NuGet

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
```

8.3 DbContext

```
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public DbSet<Libro> Libros { get; set; }
    public DbSet<Cliente> Clientes { get; set; }
    public DbSet<Pedido> Pedidos { get; set; }
    public DbSet<DetallePedido> DetallesPedido { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder options)
    {
        options.UseSqlServer("Server=localhost;Database=MxBD;Trusted_Connection=true;TrustSe");
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Configuración avanzada de relaciones
        modelBuilder.Entity<Cliente>()
            .HasMany(c => c.Pedidos)
            .WithOne(p => p.Cliente)
            .HasForeignKey(p => p.IdCliente)
            .OnDelete(DeleteBehavior.Cascade);

        // Índices y restricciones
        modelBuilder.Entity<Cliente>()
```

```

        .HasIndex(c => c.Email)
        .IsUnique();

        // Valores por defecto
        modelBuilder.Entity<Cliente>()
            .Property(c => c.FechaRegistro)
            .HasDefaultValueSql("GETDATE()");
    }
}

```

Nota importante: en el entorno real de Espiral MS, la cadena de conexión NO se hardcodea así. Se lee desde appsettings.json mediante IConfiguration inyectado (esto lo verás con ASP.NET Core en las prácticas).

8.4 Definir las entidades (modelos)

```

public class Cliente
{
    public int IdCliente { get; set; }
    public string NombreCliente { get; set; } = string.Empty;
    public string? Email { get; set; }
    public string Pais { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public DateTime FechaRegistro { get; set; }

    // Propiedad de navegación: relación uno a muchos
    public ICollection<Pedido> Pedidos { get; set; } = new List<Pedido>();
}

public class Pedido
{
    public int IdPedido { get; set; }
    public int IdCliente { get; set; }
    public DateTime FechaPedido { get; set; }
    public decimal Total { get; set; }

    // Propiedad de navegación: relación inversa
    public Cliente Cliente { get; set; } = null!;
    public ICollection<DetallePedido> Detalles { get; set; } = new List<DetallePedido>();
}

public class DetallePedido
{
    public int IdDetalle { get; set; }
    public int IdPedido { get; set; }
    public int IdLibro { get; set; }
    public int Cantidad { get; set; }
    public decimal PrecioUnitario { get; set; }

    public Pedido Pedido { get; set; } = null!;
    public Libro Libro { get; set; } = null!;
}

```

8.5 CRUD completo con un servicio

```
public class LibroServicio
{
    private readonly AppDbContext _contexto;

    public LibroServicio(AppDbContext contexto)
    {
        _contexto = contexto;
    }

    // CREATE
    public async Task<int> CrearLibroAsync(Libro libro)
    {
        _contexto.Libros.Add(libro);
        await _contexto.SaveChangesAsync();
        return libro.Id;
    }

    // READ - por Id
    public async Task<Libro?> ObtenerPorIdAsync(int id)
    {
        return await _contexto.Libros.FindAsync(id);
    }

    // READ - todos
    public async Task<List<Libro>> ObtenerTodosAsync()
    {
        return await _contexto.Libros.ToListAsync();
    }

    // READ - con filtro LINQ (idéntico a lo que ya sabes)
    public async Task<List<Libro>> ObtenerDisponiblesAsync()
    {
        return await _contexto.Libros
            .Where(l => !l.Prestado)
            .OrderBy(l => l.Titulo)
            .ToListAsync();
    }

    // UPDATE
    public async Task ActualizarAsync(Libro libro)
    {
        _contexto.Libros.Update(libro);
        await _contexto.SaveChangesAsync();
    }

    // DELETE
    public async Task EliminarAsync(int id)
    {
        var libro = await _contexto.Libros.FindAsync(id);
        if (libro != null)
        {

```

```

        _contexto.Libros.Remove(libro);
        await _contexto.SaveChangesAsync();
    }
}

```

Comparación directa con tu repositorio actual: fíjate que `ObtenerDisponiblesAsync` usa exactamente el mismo LINQ (`Where`, `OrderBy`) que ya escribiste en `BibliotecaRepositorio.ObtenerDisponibles()`. La única diferencia es `await + ToListAsync()` en vez de `ToList()`, porque ahora la consulta viaja a una base de datos real en vez de trabajar en memoria.

8.6 Consultas complejas: Include, proyecciones y agregaciones

```

// Eager loading: cargar relaciones con Include
var clienteConPedidos = await _contexto.Clientes
    .Include(c => c.Pedidos)
    .ThenInclude(p => p.Detalles)
    .FirstOrDefaultAsync(c => c.IdCliente == 1);

// Proyección con Select (traer solo lo necesario, más eficiente)
var resumenClientes = await _contexto.Clientes
    .Where(c => c.Activo)
    .Select(c => new
    {
        c.NombreCliente,
        TotalPedidos = c.Pedidos.Count,
        GastoTotal = c.Pedidos.Sum(p => p.Total)
    })
    .OrderByDescending(c => c.GastoTotal)
    .ToListAsync();

// GroupBy traducido a SQL real
var clientesPorPais = await _contexto.Clientes
    .GroupBy(c => c.Pais)
    .Select(g => new { Pais = g.Key, Total = g.Count() })
    .ToListAsync();

// Combinar Where + Join explícito (cuando Include no basta)
var pedidosRecientes = await _contexto.Pedidos
    .Where(p => p.FechaPedido >= DateTime.Now.AddDays(-30))
    .Join(_contexto.Clientes,
        pedido => pedido.IdCliente,
        cliente => cliente.IdCliente,
        (pedido, cliente) => new { cliente.NombreCliente, pedido.Total, pedido.FechaPedido })
    .ToListAsync();

```

8.7 Migraciones — versionar la base de datos

Las migraciones son como “commits de Git pero para el esquema de la BD”: cada cambio en tus clases se traduce en un script versionado.

```
# Primera migración: crea todas las tablas según tus entidades
dotnet ef migrations add InitialCreate

# Tras modificar una entidad (por ejemplo, añadir una propiedad)
dotnet ef migrations add AnadirCampoEmail

# Aplicar los cambios pendientes a la base de datos real
dotnet ef database update

# Revertir a una migración anterior
dotnet ef database update NombreMigracionAnterior

# Ver el SQL que generaría una migración sin aplicarlo
dotnet ef migrations script
```

Archivo de migración generado automáticamente:

```
public partial class AnadirCampoEmail : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<string>(
            name: "Email",
            table: "Clientes",
            type: "nvarchar(max)",
            nullable: true);
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        // Revertir el cambio, por si haces rollback
        migrationBuilder.DropColumn(name: "Email", table: "Clientes");
    }
}
```

Por qué importa en Espiral MS: cuando trabajes en equipo, cada desarrollador genera sus propias migraciones al cambiar el modelo. Git las versiona igual que el código, y `dotnet ef database update` las aplica en orden. Nadie edita la BD a mano.

8.8 Async en toda la cadena — por qué importa

Fíjate que **todos** los métodos de EF Core que tocan la base de datos terminan en Async y se usan con `await`: `ToListAsync()`, `FindAsync()`, `SaveChangesAsync()`, `FirstOrDefaultAsync()`. Esto es consistente con lo que aprendiste en la Lección 3 — cualquier operación de I/O (y una consulta a BD lo es) debe ser asíncrona para no bloquear el hilo mientras espera respuesta del servidor de base de datos.

8.9 Ejercicios Lección 8

1. Crea un `AppDbContext` con `DbSet<Libro>`, `DbSet<Cliente>`, `DbSet<Pedido>`.
2. Configura la relación uno-a-muchos entre `Cliente` y `Pedido` en `OnModelCreating`.
3. Escribe un `LibroServicio` con los 5 métodos CRUD mostrados arriba.

4. Genera la migración inicial y aplícala con `dotnet ef database update`.
 5. Escribe una consulta con Include que traiga un cliente junto a todos sus pedidos.
 6. Escribe una proyección con Select que devuelva solo nombre de cliente y su gasto total.
-

Lección 9: Git Avanzado

[↑ Volver al índice](#)

9.1 Git Flow — el modelo de ramas estándar en empresa

```
main (producción)
├─ release/v1.0
├─ develop (integración)
│   ├── feature/login
│   ├── feature/reportes
│   └─ hotfix/bug-critico
```

Ramas principales: - main: código en producción, solo recibe merges desde release o hotfix - develop: rama de integración, donde confluyen todas las features terminadas - feature/*: una rama por cada funcionalidad nueva, nace y muere en develop - hotfix/*: corrección urgente directamente sobre main (bug crítico en producción) - release/*: preparación de una versión antes de pasarla a main (últimos ajustes, testing)

9.2 Comandos básicos de repaso

```
git init                # crear un repo nuevo
git clone <url>          # clonar un repo remoto
git status              # ver el estado actual (cambios, staging)
git add archivo.cs      # añadir un archivo al staging
git add .               # añadir todos los cambios
git commit -m "mensaje" # crear un commit
git push origin nombre-rama # subir cambios al remoto
git pull origin nombre-rama # traer y fusionar cambios del remoto
git log --oneline       # historial compacto de commits
```

9.3 Crear y trabajar con ramas

```
git branch -a           # ver todas las ramas (locales y remotas)
git checkout develop
git pull origin develop
git checkout -b feature/autenticacion-usuarios # crear y cambiar a una rama nueva

# Hacer cambios y commits
git add .
git commit -m "feat: implementar login con JWT"
git commit -m "fix: validar email en registro"

# Enviar la rama al servidor
```

```
git push origin feature/autenticacion-usuarios
```

Sintaxis moderna equivalente a checkout -b

```
git switch -c feature/autenticacion-usuarios
```

Convención de mensajes de commit (Conventional Commits)

Un estándar muy usado en empresas — cada mensaje empieza con un prefijo que indica el tipo de cambio:

Prefijo	Significado
feat:	Nueva funcionalidad
fix:	Corrección de un bug
refactor:	Cambio interno sin alterar comportamiento
docs:	Solo documentación
test:	Añadir o modificar tests
chore:	Tareas de mantenimiento (dependencias, config)

```
git commit -m "feat: añadir endpoint de exportación a PDF"
```

```
git commit -m "fix: corregir cálculo de IVA en el total del pedido"
```

9.4 Pull Requests (PRs) — el flujo real en equipo

Un pull request es una solicitud para fusionar tu rama en otra (normalmente develop).

Proceso típico en empresa: 1. Terminas tu feature en feature/autenticacion 2. `git push origin feature/autenticacion` 3. En GitHub/GitLab/Azure DevOps, creas un PR describiendo qué cambia y por qué 4. Asignas un revisor (code review) 5. El revisor comenta, aprueba, o pide cambios 6. Una vez aprobado, se mergea a develop (normalmente con “squash and merge” para mantener el historial limpio) 7. La rama feature se elimina

Buenas prácticas al abrir un PR: - Título claro y descriptivo, siguiendo la misma convención que los commits - Descripción de qué se hizo y por qué (no solo “arreglé un bug”) - PRs pequeños y enfocados — más fáciles de revisar que uno gigante con 20 cambios distintos - Enlazar el ticket/issue relacionado si el equipo usa Jira o similar

9.5 Mergear ramas localmente

```
git checkout develop
```

```
git pull origin develop
```

```
git merge feature/autenticacion
```

Si hay conflictos, Git te lo indica:

CONFLICT (content): Merge conflict in Libro.cs

```
git status    # ver qué archivos tienen conflicto
```

Abrir el archivo y buscar los marcadores de conflicto:

<<<<<< HEAD

```
# tu código actual en develop
# =====
# el código de la rama que estás mergeando
# >>>>>> feature/autenticacion

# Tras resolver manualmente y elegir qué código conservar:
git add Libro.cs
git commit -m "merge: resolver conflictos en Libro.cs"
git push origin develop
```

9.6 Rebase — alternativa más limpia a merge

```
git checkout feature/autenticacion
git rebase develop           # reaplica tus commits encima de los últimos cambios de develop

# Si hay conflictos durante el rebase:
git add archivo-resuelto.cs
git rebase --continue

# Si te arrepientes a mitad del rebase:
git rebase --abort

# Como el rebase reescribe el historial, el push normal falla; hace falta forzar:
git push origin feature/autenticacion --force-with-lease
```

--force-with-lease en vez de --force: es más seguro porque falla si alguien más subió cambios a esa rama remota que tú no tienes localmente, evitando sobrescribir el trabajo de otra persona por accidente.

Merge vs Rebase — cuándo usar cada uno: - **Merge:** conserva el historial exacto de cómo ocurrieron las cosas, incluye un commit de merge. Más seguro para ramas compartidas por varias personas. - **Rebase:** historial más limpio y lineal, sin commits de merge extra. Ideal para tu propia rama feature antes de abrir el PR, nunca sobre una rama que otros ya están usando.

9.7 Deshacer cambios

```
git restore archivo.cs           # descartar cambios no confirmados en un archivo
git restore --staged archivo.cs  # sacar un archivo del staging (sin perder el cambio)
git reset --soft HEAD~1         # deshacer el último commit, conservando los cambios
git reset --hard HEAD~1         # deshacer el último commit, DESCARTANDO los cambios
git revert <hash-del-commit>    # crear un commit nuevo que deshace uno anterior
```

Regla de oro: en una rama compartida con el equipo, usa siempre revert (crea un commit nuevo) en vez de reset (reescribe el historial) — así nadie más se rompe al hacer pull.

9.8 Etiquetas (tags) para versiones

```
git tag -a v1.0.0 -m "Release version 1.0.0"
git push origin v1.0.0
git tag # listar todas las etiquetas
git checkout v1.0.0 # ver el código exacto de esa versión
```

9.9 Buscando en el historial

```
git log archivo.cs # commits que tocaron un archivo concreto
git log --author="tu-nombre" # commits de una persona
git log --since="2026-01-01" --until="2026-02-01" # commits en un rango de fechas
git blame archivo.cs # quién cambió cada línea y en qué commit
git diff develop feature/autenticacion # diferencias entre dos ramas
```

9.10 .gitignore — qué no subir al repositorio

```
# .gitignore típico para un proyecto .NET
bin/
obj/
*.user
.vs/
appsettings.Development.json
```

Nunca se sube al repositorio: binarios compilados (bin/, obj/), configuración local con credenciales, ni archivos temporales del IDE.

9.11 Ejercicios Lección 9

1. Crea una rama feature/nueva-funcionalidad desde develop
 2. Haz 3 commits con mensajes siguiendo Conventional Commits
 3. Cambia a develop y haz un commit ahí también (para forzar un conflicto)
 4. Mergea feature/nueva-funcionalidad a develop y resuelve el conflicto
 5. Crea una etiqueta v0.1.0 en develop
 6. Practica git rebase en una rama feature nueva antes de mergearla
 7. Simula un error: haz un commit, luego deshazlo con git revert
 8. Crea un .gitignore apropiado para un proyecto C#/.NET
-

Lección 10: Scrum y Agile

[↑ Volver al índice](#)

10.1 ¿Qué es Scrum?

Scrum es un marco de trabajo ágil para gestionar proyectos complejos mediante ciclos cortos e iterativos llamados **sprints**. En vez de planificar todo el proyecto de golpe (modelo “cascada”), el equipo entrega software funcionando cada 1-2 semanas y ajusta el rumbo según feedback real.

Por qué se usa en empresas de software: los requisitos cambian, y Scrum asume eso desde el diseño en vez de luchar contra ello.

10.2 Roles

Rol	Responsabilidad
Product Owner (PO)	Define qué construir, prioriza el backlog, acepta o rechaza el trabajo terminado
Scrum Master	Facilita el proceso, elimina obstáculos, protege al equipo de interrupciones externas
Development Team	Implementa las features, se auto-organiza, típicamente 5-8 personas

Importante: el Scrum Master no es un jefe de proyecto tradicional — no asigna tareas, facilita que el equipo se organice solo.

10.3 Artefactos (Artifacts)

Artefacto	Qué es	Ejemplo
Product Backlog	Lista completa de todo lo pendiente, ordenada por prioridad	Login, reportes, API de pagos, exportar a PDF...
Sprint Backlog	Subconjunto del backlog que el equipo se compromete a hacer en este sprint	5-6 items para las próximas 2 semanas
Increment	El resultado tangible: software funcionando y probado al final del sprint	Versión desplegable con las nuevas features

10.4 El ciclo de un Sprint

Un sprint dura típicamente **1-2 semanas** (2 semanas es lo más común en entornos empresariales, incluido probablemente Espiral MS).

Sprint Planning (inicio del sprint)

Duración: 2-4 horas. El equipo y el PO seleccionan y desglosan los items del backlog.

PO: "Necesitamos poder exportar el catálogo de libros a PDF"

Team desglosa:

- Investigar librería de generación de PDF (2 pts)
- Crear endpoint /api/libros/exportar (3 pts)
- Diseñar plantilla del PDF (2 pts)
- Tests de integración (1 pt)

Total: 8 puntos. El equipo se compromete a esto para el sprint.

Daily Standup (cada día, 15 minutos máximo)

Cada persona responde brevemente: - ¿Qué hice ayer? - ¿Qué haré hoy? - ¿Tengo algún obstáculo (blocker)?

Dev: "Ayer terminé el endpoint de exportación.

Hoy empiezo la plantilla del PDF.

Obstáculo: necesito acceso a la librería iTextSharp, aún no la tengo instalada."

Scrum Master: "Anotado, te ayudo con el acceso después de la reunión."

Regla de oro: el standup no es para resolver problemas en el momento, sino para detectarlos. La resolución ocurre después, fuera de la reunión.

Sprint Review (final del sprint, 1-2 horas)

El equipo hace una **demo en vivo** de lo que terminó, ante el PO y stakeholders. No son diapositivas — es software funcionando.

Sprint Retrospective (final del sprint, 1 hora)

El equipo reflexiona sobre el propio proceso de trabajo (no sobre el producto):

Qué salió bien:

- La comunicación entre backend y frontend fue fluida
- Detectamos un bug crítico gracias a los tests automatizados

Qué no salió bien:

- Subestimamos la complejidad de la plantilla PDF (2 pts se convirtieron en 5)
- El PO no estuvo disponible dos días para resolver dudas

Acciones para el próximo sprint:

- Reservar 30 min diarios de disponibilidad del PO
- Añadir un margen de contingencia al estimar tareas con librerías nuevas

10.5 Story Points y estimación

Scrum no estima en horas exactas, sino en **complejidad relativa** usando Story Points. Escala típica (Fibonacci): **1, 2, 3, 5, 8, 13, 21**

Puntos	Complejidad aproximada
1	Trivial — menos de 1 hora (cambiar un texto, un color)
2	Pequeño — 2-4 horas (endpoint GET simple sin lógica)
3	Pequeño-medio — 4-8 horas (endpoint con validaciones)
5	Medio — 1-2 días (feature con lógica de negocio moderada)
8	Grande — 2-3 días (feature con BD, validaciones y tests)
13+	Muy grande — debe desglosarse en tareas más pequeñas

Planning Poker (técnica de estimación en grupo)

1. El PO explica la feature
2. Cada desarrollador estima en secreto (con cartas físicas o una app)
3. Todos revelan su estimación a la vez
4. Si hay diferencias grandes, se discute el porqué y se re-estima

Feature: "Dashboard de estadísticas de préstamos"

Dev A: 5 puntos → "Pensé que era solo una tabla con datos"

Dev B: 8 puntos → "Pero hay que agregar por fechas, cachear resultados y hacer gráficos"

Dev A: "Ah, no había pensado en el caching. Cambio a 8."

Consenso final: 8 puntos

Este proceso es valioso no tanto por el número final, sino porque **obliga a discutir la complejidad real antes de empezar a programar.**

10.6 Velocidad del equipo (Velocity)

La **velocidad** es la cantidad de story points que el equipo completa, de media, por sprint. Se usa para planificar sprints futuros con datos reales en vez de suposiciones.

Sprint 1: comprometidos 20 pts, completados 18 pts

Sprint 2: comprometidos 22 pts, completados 21 pts

Sprint 3: comprometidos 20 pts, completados 19 pts

Velocidad media: ~19-20 puntos por sprint

→ Para el Sprint 4, el equipo no debería comprometerse a más de ~20 puntos, aunque el backlog tenga 35 puntos de features "deseables".

10.7 Tablero Kanban (herramienta de seguimiento visual)

Muchos equipos Scrum usan un tablero Kanban (en Jira, Azure DevOps o Trello) para visualizar el flujo del sprint:

TO DO	IN PROGRESS	REVIEW	DONE
Exportar PDF (5 pts)	Endpoint login (Dev1, en curso)	Validar JWT (Dev2, esperando aprobación)	Crear BD usuarios (- completado)
Plantilla PDF (2 pts)			Generar JWT (- completado)

El equipo mueve las tarjetas de izquierda a derecha durante el sprint. Un vistazo al tablero dice inmediatamente dónde está el cuello de botella.

10.8 Qué esperar en Espiral MS

Con alta probabilidad, en las prácticas: - Sprints de 2 semanas, con Jira o Azure DevOps para el backlog - Daily standup a primera hora de la mañana - Sprint Planning el primer día del sprint, Review/Retro el último - Se te pedirá estimar tus propias tareas en story points

Consejo práctico: al principio, tus estimaciones probablemente sean optimistas (es normal, le pasa a todo el mundo). Lo importante no es acertar siempre, sino comunicar pronto cuando algo se complica más de lo esperado — eso es lo que un equipo Scrum realmente valora.

10.9 Ejercicio práctico

Simula un mini-sprint sobre tu proyecto de biblioteca:

1. Escribe 5 features nuevas para el Gestor de Biblioteca (ej: “renovar préstamo”, “historial de un usuario”, “notificar por email al devolver”)
 2. Estima cada una en story points usando la tabla de arriba
 3. Selecciona 3-4 que sumen entre 10-13 puntos (tu “sprint”)
 4. Implementa una de ellas
 5. Escribe 2 líneas de “retrospectiva”: qué salió bien, qué mejorarías
-

Lección 11: Frontend

[↑ Volver al índice](#)

11.1 HTML5 Semántico

Tener nociones básicas de HTML no es lo mismo que escribirlo con **semántica** — usar la etiqueta que realmente describe el contenido, no solo `<div>` para todo.

```
<!-- MAL: sin semántica, todo son divs -->
<div>
  <div>Mi Blog</div>
  <div>
    <div><a href="/">Home</a></div>
  </div>
</div>

<!-- BIEN: cada etiqueta describe su función -->
<header>
  <h1>Mi Blog</h1>
  <nav>
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/about">About</a></li>
    </ul>
  </nav>
</header>

<main>
  <article>
    <h2>Primer post</h2>
    <p>Contenido del artículo...</p>
    <footer>
      <time datetime="2026-07-12">12 de julio de 2026</time>
    </footer>
  </article>
</main>

<aside>
  <h3>Sidebar</h3>
  <p>Contenido secundario...</p>
</aside>
```

```
<footer>
  <p>&copy; 2026 Mi Blog</p>
</footer>
```

Tags semánticos clave

Tag	Uso
<header>	Cabecera de página o de una sección
<nav>	Bloque de navegación
<main>	Contenido principal (uno solo por página)
<article>	Contenido independiente y reutilizable
<section>	Agrupación de contenido relacionado
<aside>	Contenido lateral o secundario
<figure> + <figcaption>	Imagen con leyenda
<time>	Fecha u hora, en formato legible por máquina

Por qué importa: los lectores de pantalla (accesibilidad) y los motores de búsqueda entienden mucho mejor la estructura de la página con etiquetas semánticas que con <div> genéricos.

11.2 Formularios HTML5

```
<form>
  <label for="email">Correo electrónico:</label>
  <input type="email" id="email" name="email" required>

  <label for="edad">Edad:</label>
  <input type="number" id="edad" name="edad" min="18" max="120">

  <label for="fecha">Fecha de nacimiento:</label>
  <input type="date" id="fecha" name="fecha">

  <button type="submit">Enviar</button>
</form>
```

Los tipos email, number, date activan validación nativa del navegador sin escribir JavaScript.

11.3 CSS3 — Flexbox y Grid

Flexbox (alineación en una dimensión)

```
.contenedor {
  display: flex;
  justify-content: space-between; /* alineación horizontal */
  align-items: center;           /* alineación vertical */
  gap: 20px;                    /* espacio entre elementos */
  flex-wrap: wrap;              /* permite que los items salten de línea */
}
```

```
.item {
  flex: 1; /* crece proporcionalmente para llenar el espacio disponible */
}
```

Grid (alineación en dos dimensiones)

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 columnas de igual ancho */
  grid-template-rows: auto;
  gap: 20px;
}

.item-destacado {
  grid-column: span 2; /* ocupa 2 columnas */
  grid-row: span 2; /* ocupa 2 filas */
}
```

Cuándo usar cada uno: Flexbox para alinear elementos en una fila o columna (una navbar, una lista de tarjetas en línea). Grid para layouts de página completos con filas y columnas a la vez.

Responsive design con media queries

```
/* Móvil primero (mobile-first) */
.header {
  font-size: 18px;
  padding: 10px;
}

/* Tablets */
@media (min-width: 768px) {
  .header {
    font-size: 24px;
    padding: 20px;
  }
}

/* Desktop */
@media (min-width: 1024px) {
  .header {
    font-size: 32px;
  }
}
```

11.4 JavaScript Moderno (ES6+)

Arrow functions

```
// Sintaxis clásica
function sumar(a, b) {
  return a + b;
}
```

```

}

// Arrow function equivalente
const sumar = (a, b) => a + b;

// Con cuerpo de varias líneas
const procesar = (dato) => {
  console.log("Procesando...");
  return dato * 2;
};

```

Destructuring

```

// De arrays
const [nombre, edad, ciudad] = ["Ana", 30, "Madrid"];

// De objetos
const usuario = { id: 1, email: "ana@example.com" };
const { id, email } = usuario;

// En parámetros de función directamente
const mostrar = ({ id, email }) => console.log(`${id}: ${email}`);

```

Template literals

```

const nombre = "Juan";
const edad = 25;

// Antes (concatenación manual)
console.log("Hola, " + nombre + ". Tienes " + edad + " años.");

// Ahora (template literal)
console.log(`Hola, ${nombre}. Tienes ${edad} años.`);

```

Spread y Rest operators

```

// Spread: expandir un array/objeto
const numeros = [1, 2, 3];
const masNumeros = [...numeros, 4, 5]; // [1, 2, 3, 4, 5]

const persona = { nombre: "Ana" };
const personaCompleta = { ...persona, edad: 30 }; // añade edad sin mutar el original

// Rest: agrupar argumentos sobrantes
function sumarTodos(...numeros) {
  return numeros.reduce((acc, n) => acc + n, 0);
}
sumarTodos(1, 2, 3, 4); // 10

```

async/await en JavaScript

```
// Antes (promesas encadenadas)
fetch("/api/usuarios")
  .then(response => response.json())
  .then(usuarios => console.log(usuarios))
  .catch(error => console.error(error));

// Ahora (async/await, igual de concepto que en C#)
async function obtenerUsuarios() {
  try {
    const response = await fetch("/api/usuarios");
    const usuarios = await response.json();
    console.log(usuarios);
  } catch (error) {
    console.error(error);
  }
}
```

Array methods (el equivalente JS de LINQ)

```
const numeros = [1, 2, 3, 4, 5];

const pares = numeros.filter(n => n % 2 === 0);           // como .Where()
const cuadrados = numeros.map(n => n * n);                // como .Select()
const primero = numeros.find(n => n > 2);                 // como .FirstOrDefault()
const suma = numeros.reduce((acc, n) => acc + n, 0);       // agregación
const hayAlguno = numeros.some(n => n > 4);               // como .Any()
const todosPositivos = numeros.every(n => n > 0);         // como .All()
```

11.5 Ejercicio práctico

1. Crea una página HTML con estructura semántica completa (header, nav, main, article, aside, footer)
2. Usa Flexbox para una barra de navegación y Grid para una galería de tarjetas
3. Añade media queries para que se vea bien en móvil y en desktop
4. Escribe JavaScript que obtenga datos de <https://jsonplaceholder.typicode.com/users> con fetch + async/await
5. Muestra los usuarios dinámicamente en la página usando .map() para generar el HTML
6. Añade un campo de texto que filtre los usuarios por nombre en tiempo real usando .filter()

Lección 12: TypeScript

[↑ Volver al índice](#)

12.1 ¿Qué es TypeScript y por qué usarlo?

TypeScript es JavaScript con **tipos estáticos**, desarrollado por Microsoft. Compila a JavaScript normal — el navegador nunca ejecuta TypeScript directamente.

```
// JavaScript: sin tipos, el error solo aparece en tiempo de ejecución
function sumar(a, b) {
  return a + b;
}
sumar("5", 3); // "53" – concatenó en vez de sumar, bug silencioso

// TypeScript: el compilador detecta el error ANTES de ejecutar
function sumar(a: number, b: number): number {
  return a + b;
}
sumar("5", 3); // Error de compilación: string no assignable a number
```

Por qué importa en Espiral MS: si el frontend usa Angular o un stack TypeScript, esta detección temprana de errores ahorra muchísimo tiempo de debugging comparado con JavaScript puro.

12.2 Tipos básicos

```
const nombre: string = "Ana";
const edad: number = 30;
const activo: boolean = true;

// any: evita el tipado, úsalo solo como último recurso
let cualquiera: any = "hola";
cualquiera = 123; // válido, pero pierdes toda la protección de tipos

// union types: la variable puede ser uno de varios tipos
let id: number | string = 1;
id = "ABC123"; // también válido

// type alias: definir una "forma" reutilizable de objeto
type Usuario = {
  id: number;
  nombre: string;
  email?: string; // el "?" indica propiedad opcional
};

const user: Usuario = { id: 1, nombre: "Ana" }; // email es opcional, se puede omitir

// arrays tipados
const numeros: number[] = [1, 2, 3];
const nombres: Array<string> = ["Ana", "Luis"];
```

12.3 Interfaces (paralelo directo con C#)

```
interface IRepository<T> {
  obtenerTodos(): Promise<T[]>;
  obtenerPorId(id: number): Promise<T>;
  agregar(item: T): Promise<number>;
}
```

```

class RepositorioUsuarios implements IRepository<Usuario> {
  async obtenerTodos(): Promise<Usuario[]> {
    const response = await fetch("/api/usuarios");
    return response.json();
  }

  async obtenerPorId(id: number): Promise<Usuario> {
    const response = await fetch(`/api/usuarios/${id}`);
    return response.json();
  }

  async agregar(usuario: Usuario): Promise<number> {
    const response = await fetch("/api/usuarios", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(usuario)
    });
    const resultado = await response.json();
    return resultado.id;
  }
}

```

Esta estructura es literalmente el mismo patrón Repository que viste en C# (Lección 6), solo que en TypeScript.

12.4 Generics en TypeScript

```

// Función genérica
function obtenerPrimero<T>(lista: T[]): T {
  if (lista.length === 0) throw new Error("Lista vacía");
  return lista[0];
}

const numeros = [1, 2, 3];
const primero = obtenerPrimero(numeros); // tipo inferido: number

// Clase genérica
class Contenedor<T> {
  constructor(private valor: T) {}

  obtener(): T {
    return this.valor;
  }
}

const contenedor = new Contenedor<string>("Hola");

```

12.5 Enums

```
enum EstadoPedido {
    Pendiente,
    Procesando,
    Enviado,
    Entregado
}

let estado: EstadoPedido = EstadoPedido.Procesando;

// Enum con valores string (más legible en debugging/logs)
enum Rol {
    Admin = "ADMIN",
    Usuario = "USUARIO",
    Invitado = "INVITADO"
}
```

12.6 Configuración básica de un proyecto TypeScript

```
npm init -y
npm install typescript --save-dev
npx tsc --init           # genera tsconfig.json
npx tsc archivo.ts       # compila a archivo.js
npx tsc --watch          # recompila automáticamente al guardar

// tsconfig.json – opciones más relevantes
{
  "compilerOptions": {
    "target": "ES2020",
    "strict": true,        // activa todas las comprobaciones estrictas de tipos
    "outDir": "./dist",
    "module": "commonjs"
  }
}
```

12.7 Ejercicio práctico

1. Reescribe tu código JavaScript de la Lección 11 en TypeScript, añadiendo tipos explícitos
 2. Define una interfaz Usuario con id, nombre, email (opcional)
 3. Crea una interfaz genérica IRepository<T> y una clase RepositorioUsuarios que la implemente
 4. Añade un enum EstadoUsuario con valores Activo, Inactivo, Pendiente
 5. Configura un tsconfig.json con modo strict activado y compila tu código
-

Resumen Final

[↑ Volver al índice](#)

Lo que has aprendido

- **C# moderno:** Sintaxis, propiedades, null-coalescing
- **LINQ:** El equivalente y mejora de Streams de Java
- **async/await:** La base del código moderno en .NET
- **Generics:** Constraints, classes genéricas, reificación
- **Delegates/Eventos:** Patrón publisher/subscriber nativo
- **Patrones:** Repository, DI, Factory, Singleton
- **SQL:** Oracle SQL y T-SQL
- **Entity Framework:** LINQ → SQL automático
- **Git Flow:** Ramas, merge, pull requests profesionales
- **Scrum:** Sprints, story points, ceremonias
- **Frontend:** HTML5, CSS3, JavaScript/TypeScript moderno

Fin de la Guía Definitiva

¡Mucho éxito en Espiral MS!